

Assignment: Create, Reset, and Rebuild the Todo Speckit App

Goal: Create your own GitHub repository from OC CS Speckit, strip the shipped Todo implementation (answer key), then rebuild the application **from the feature specs** using Spec-Driven Development (SDD) and Cursor.

Related docs: [README](#) · [STARTER-KIT.md](#) · [features/README.md](#) · [Building each feature with Cursor](#)

Not this assignment: Starting a **new** product from the starter kit — use [ASSIGNMENT-starter-kit.md](#) instead. Reading-only tour of Speckit — use [ASSIGNMENT-walkthrough-todo.md](#).

Learning outcomes

By the end of this assignment you will be able to:

1. Create a public GitHub repo and push a full Speckit tree into it.
 2. Separate **scaffold** (`main`) from **integration** (`dev`) and **feature** branches.
 3. Use `npm run reset:example` to remove product code while keeping specs, rules, and ADRs.
 4. Implement Features 1–5 from `features/feature-*.md` with Cursor, tests, and living reference updates.
-

Prerequisites

Tool	Notes
Node.js 24+	<code>node -v</code>
npm	Comes with Node
MySQL	XAMPP, WAMP, or native MySQL
Git + GitHub account	Browser + git CLI
Cursor	Agent mode for implementation

Upstream (instructor / org repo): `https://github.com/OC-ComputerScience/todospeckit.git`
(If your course uses a different URL or release tag, use that instead.)

Part A — Create your GitHub repository

1. Open [GitHub → New repository](#).
 2. Set:
 - **Repository name:** e.g. `todo-speckit` or `todospeckit-<your-username>`
 - **Visibility:** Public
 - **Do not** add a README, `.gitignore`, or license (keep it empty so the first push is clean)
 3. Create the repository. Copy the HTTPS or SSH URL, e.g. `https://github.com/<you>/todo-speckit.git`
-

Part B — Load the repo with todo-speckit

Start from a **ZIP download** so your local repo has a fresh git history (not a clone or fork of upstream).

1. Open the upstream repo: [OC-ComputerScience/todospeckit](#).
2. Click **Code** → **Download ZIP** (or use a course release ZIP if your instructor provides one).

3. Unzip into a folder named `todo-speckit` (or rename the unzipped folder).
4. Open the unzipped folder in **Cursor**: File → Open Folder... (macOS and Windows). Optionally, from a system terminal inside the folder run `cursor .` if the Cursor CLI is installed. Work in **IDE / Agent** mode for the rest of this assignment.
5. Open a terminal in Cursor (Terminal → New Terminal) and enter the commands listed below. The terminal starts in the project folder, so you do not need to `cd .` Replace `<my-repo>` with the full GitHub URL you copied in Part A step 3 (e.g. `https://github.com/<you>/todo-speckit.git`):

```
git init
git add .
git commit -m "Initial import: OC CS Speckit Todo example"
git branch -M main
git remote add origin <my-repo>
git push -u origin main
```

Confirm on GitHub that your repo has the full tree (`features/` , `frontend/` , `backend/` , `.cursor/rules/` , etc.).

Part C — Local setup (before stripping code)

Do this once so env and DBs exist before and after reset.

Platform note: `git` and `npm` commands are the same on macOS and Windows. Where a step copies files or chains `cd` with a command, **macOS / Linux** and **Windows** variants are shown.

C1. Install dependencies

In the terminal enter these commands:

```
npm install --prefix frontend
npm install --prefix backend
npm install # root - PDF / tooling scripts (optional for rebuild)
```

C2. Create MySQL databases

Use **phpMyAdmin** (e.g. via XAMPP: <http://localhost/phpmyadmin>) or **MySQL Workbench** to create these two databases:

```
CREATE DATABASE `todospeckit-db`;
CREATE DATABASE `todospeckit-db-test`;
```

In phpMyAdmin: open the SQL tab (or use New → database name) and run the statements above.

In MySQL Workbench: open a query tab connected to your local server and run the same statements.

C3. Configure environment

A `.env` file holds **local secrets and settings** (MySQL host/user/password, database name, JWT secret, port) that the backend reads at startup. These values differ per machine, so they are **not** checked into git — you copy from the checked-in `.env.example` templates instead.

In the terminal, copy the example files (use the block for your OS):

macOS / Linux

```
cp backend/.env.example backend/.env
cp backend/.env.test.example backend/.env.test
```

Windows (Command Prompt or PowerShell)

```
copy backend\.env.example backend\.env
copy backend\.env.test.example backend\.env.test
```

Edit `backend/.env` and `backend/.env.test` with your MySQL user/password.

`backend/.env` is for the running app; `backend/.env.test` is for automated tests (uses `todospeckit-db-test` so tests do not wipe your dev data).

Do **not** commit `.env` files.

C4. (Optional) Sanity-check the answer key

Before stripping, you may run the shipped app once so you know the target UX. Open **two** terminals in Cursor.

macOS / Linux

```
# Terminal 1
cd backend && npm run dev

# Terminal 2
cd frontend && npm run dev
```

Windows (Command Prompt or PowerShell)

```
REM Terminal 1
cd backend
npm run dev

REM Terminal 2
cd frontend
npm run dev
```

Service	URL
Frontend	http://localhost:8082
API	http://localhost:3200/todo/

From the **repo root** (new terminal or `cd ..` back from `backend` / `frontend`):

```
npm test
```

This should pass on the answer key. (`npm` / `git` commands are the same on Mac and Windows.)

Part D — Remove all product code (keep specs)

This is the core of the assignment: turn the answer key into a **scaffold + Todo specs** baseline.

D1. Preview the reset

```
npm run reset:example -- --dry-run
```

Expect deletes of controllers, models, routes, views, services, and feature tests, plus restore of empty shells and empty `features/reference/*` stubs. Specs under `features/feature-*.md` stay.

D2. Apply the reset

```
npm run reset:example -- --yes
```

What stays: `features/feature-1 ... feature-5`, `framework/guides`, `.cursor/rules/`, ADRs, NFRs, C4 diagrams, tooling, empty Vue/Express shells.

What goes: Todo product implementation (models, API, UI, feature tests).

API prefix: still `/todo/` so specs match.

D3. Commit the scaffold on `main`

Constitution: `main` = scaffold only (no feature implementation).

Use **Cursor** to commit and push (recommended):

1. Open the **Source Control** view (left sidebar, branch icon, or `Ctrl+Shift+G` / `Cmd+Shift+G`).
2. Confirm you are on `main` (status bar at the bottom of the window). If not, click the branch name and switch to `main`.
3. Review the changed files, stage them (`+` / Stage All Changes).
4. Enter a commit message, e.g. `Reset Todo example to scaffold; keep feature specs for rebuild`, then **Commit**.
5. Click **Sync Changes** or **Publish Branch / Push** to send `main` to GitHub.

Alternatively, ask Cursor **Agent** (chat):

```
Commit all reset changes on main with message
"Reset Todo example to scaffold; keep feature specs for rebuild"
and push to origin main.
```

Or run the same steps in the Cursor terminal:

```
git checkout main # or create main from your default branch if needed
git status
git add -A
git commit -m "Reset Todo example to scaffold; keep feature specs for rebuild"
git push -u origin main
```

Optional baseline tag (terminal):

```
git tag scaffold-v1
git push origin scaffold-v1
```

D4. Create dev from main

All feature work merges into `dev`, never into `main`.

Use **Cursor** to create and publish `dev` (recommended):

1. Confirm you are on `main` (status bar).
2. Click the branch name in the status bar → **Create new branch...** → name it `dev` → create from `main`.
3. Push / publish the branch: **Publish Branch** (or Sync) so `dev` exists on GitHub.

Alternatively, ask Cursor **Agent**:

```
Create a branch named dev from main and push it to origin.
```

Or run in the Cursor terminal:

```
git checkout main
git checkout -b dev
git push -u origin dev
```

D5. Verify harness after reset

```
npm test
```

Only harness / shell tests should run and pass. Feature suites are gone until you rebuild them.

Part E — Recreate the code from specs

Implement features **in order**. Branch from `dev` each time. Use Cursor **Agent** mode; `@` mention the feature file.

Order	Spec	Branch
1	feature-1-user-auth.md	feature/1-user-auth
2	feature-2-todo-list-management.md	feature/2-todo-list-management
3	feature-3-todo-list-item-management.md	feature/3-todo-list-item-management
4	feature-4-user-profile-management.md	feature/4-user-profile-management
5	feature-5-todo-due-date.md	feature/5-todo-due-date

Features 4 and 5 both depend on 1–3; they do not depend on each other (you may do either after Feature 3).

E1. Per-feature git workflow

Never merge feature branches into `main`.

Start a feature branch

Use **Cursor** (recommended):

1. Click the branch name in the status bar → switch to `dev` (pull/sync first if needed so `dev` is up to date).
2. Click the branch name again → **Create new branch...** → enter the exact name from the table above (e.g. `feature/1-user-auth`) → create from `dev`.
3. Optionally **Publish Branch** so the feature branch exists on GitHub.

Alternatively, ask Cursor **Agent**:

```
Switch to dev, pull the latest, and create branch feature/1-user-auth from dev.
```

(Use the exact branch name for the feature you are starting.)

Or in the Cursor terminal:

```
git checkout dev
git pull origin dev
git checkout -b feature/N-short-name # exact name from the table above
```

Implement the code - instruct the agent

1. Enter the command: Implement feature @feature-1-user-auth.md (for example)
2. Watch the agent plan, implement and test the code.

User test the code

if changes were made in the backend code. Restart the backing and test the applicaiotn to see how the featuer was implemented.

After implementation — commit and merge into `dev`

1. In **Source Control**, stage changes, commit with a clear message, and push the feature branch.
2. Merge into `dev` via a GitHub **Pull Request** (feature branch → `dev`), or locally:

Ask Cursor **Agent**:

```
Commit my feature work, push this feature branch, then merge it into dev and push origin dev.
Do not merge into main.
```

Or in the terminal:

```
git checkout dev
git merge feature/N-short-name
git push origin dev
```

E2. Cursor prompt (full feature)

On the correct `feature/*` branch:

```
Implement @features/feature-N-....md per its Agent implementation request and Definition of Done.
```

E3. Micro-step order (every feature)

This how the agent implements the feature.

Step	Ask Cursor for	Verify
1	Backend models + associations	Backend starts (cd backend then npm run dev)
2	Routes + controllers + auth helpers	API responds
3	Backend tests (Gherkin / Test Coverage Map)	npm run test:backend
4	Frontend *Services.js + axios client	No axios in views
5	Views + components	Frontend starts (cd frontend then npm run dev)
6	Frontend tests	npm run test:frontend
7	Router / e2e manual check	Browser flow works

After each feature: `npm test` from repo root must pass. Update `features/reference/api.md` , `data-model.md` , and/or `behavior.md` in the **same** PR when schema or API changes.

Layer-by-layer example prompts: [README — Building each feature with Cursor](#).

E4. Definition of Done (each feature)

A feature is done when:

- ☐ Every user story and **FR-00N** has matching code
- ☐ **SC-00N** success criteria are met
- ☐ Every Gherkin scenario has ≥ 1 automated test (Test Coverage Map)
- ☐ `npm test` passes
- ☐ Living reference updated when API/schema/behavior changed
- ☐ Manual demo works in the browser
- ☐ Merged into `dev` (not `main`)

E5. Manual acceptance smoke (after Features 1–3)

1. Register → log in → stay signed in after refresh → log out.
2. Create / rename / delete todo lists on the dashboard.
3. Add / complete / edit / delete todos; switch lists; deleting a list removes its todos.
4. (Feature 4) Edit profile from the menu.
5. (Feature 5) Set due dates; overdue styling appears as specified.

Part F — Deliverables

1. **Canvas:** Submit your **GitHub repository link** in the text of the Canvas assignment.
2. **Demo:** Demo your working system to the professor or GA (register/login, lists, todos, and any other features required for your section).

Your repo should show:

Branch / artifact	Expected
-------------------	----------

main	Scaffold after <code>reset:example</code> (specs + empty shells, no feature product code)
dev	Completed features merged; <code>npm test</code> passes
Feature branches / PRs	One branch per feature into dev (not into main)

Suggested commit milestones

1. Initial import / first push of Speckit
2. Reset Todo example to scaffold; keep feature specs on main
3. Create dev from main
4. One merge (or PR) per feature into dev

Quick command cheat sheet

`git` and `npm` commands are the same on **macOS** and **Windows**. Only file-copy and some `cd` chaining differ.

```
# After Download ZIP, open the folder in Cursor, then in the Cursor terminal:
git init
git add .
git commit -m "Initial import: OC CS Speckit Todo example"
git branch -M main
git remote add origin <my-repo>
git push -u origin main

# Env (macOS / Linux)
npm install --prefix frontend && npm install --prefix backend
cp backend/.env.example backend/.env
cp backend/.env.test.example backend/.env.test

# Env (Windows) – use these instead of cp:
# npm install --prefix frontend
# npm install --prefix backend
# copy backend\.env.example backend\.env
# copy backend\.env.test.example backend\.env.test
# CREATE DATABASE todospeckit-db; CREATE DATABASE todospeckit-db-test;

# Strip answer-key code
npm run reset:example -- --dry-run
npm run reset:example -- --yes
git add -A && git commit -m "Reset Todo example to scaffold; keep feature specs"
git push origin main
git checkout -b dev && git push -u origin dev

# Rebuild Feature N
git checkout dev && git checkout -b feature/1-user-auth
# ... Cursor: Implement @features/feature-1-user-auth.md per Agent implementation request ...
npm test
git checkout dev && git merge feature/1-user-auth && git push origin dev
```

Troubleshooting

Problem	Fix
Agent implements on main or dev	Stop; check out feature/N-... first (constitution + feature-branch rule)
Agent invents behavior not in the spec	Refuse; update features/ first or drop the code
Wrong stack / API shape	Cite @.cursor/rules/ (e.g. api-conventions, ui-style-system)
Tests weakened to "pass"	Require real Gherkin coverage; no expect(true).toBe(true)
reset:example refused	Pass --yes; use --dry-run first
Confused with starter zip	starter:zip = new product (no Todo specs). This assignment uses reset:example